



开始写作

记录我的学习

请教专家

向专家提问



文件

关注
发信



东航梧州飞机坠毁132人，历史上还有哪些空难



小公司团队建设思考，协调前端妹纸和IT男氛围

Centos7下安装配置keepalived

net5 webapi中 SwaggerUI如何进行版本...

让你彻底明白sql注入攻击

使用Camtasia给视频或者图片调色

【C】【第十二课】

如何使用评论

c# netcore http请求封装

B - Pie (二分) POJ - 3122

编译原理之理解语法树、短语、直接短语...

认识Python

Elasticsearch 9300无法访问,客户端出现...

python爬虫入门10分钟爬取一个网站

【luogu3403】跳楼机 [同余最短路]

loj 2336 [JOI 2017 Final] 绳

html 快速布局



技术小哥用手机实现激光笔，女友直呼：太酷了！



结婚登记人数创36年新低，结婚率下降原因是？



广告合作
发信联系

扫码查看

Google的开源C++单元测试框架Google Test

玩转Google开源C++单元测试框架Google Test系列(gtest)(总)

前段时间学习和了解了下Google的开源C++单元测试框架Google Test，简称gtest，非常的不错。我们原来使用的是自己实现的一套单元测试框架，在使用过程中，发现越来越多使用不便之处，而这样不便之处，gtest恰恰很好的解决了。

其实gtest本身的实现并不复杂，我们完全可以模仿gtest，不断的完善我们的测试框架，但最后还是决定使用gtest取代掉原来的自己的测试框架，原因是：

- 1.不断完善我们的测试框架之后就会发觉相当于把gtest重新做了一遍，虽然轮子造的很爽，但是不是必要的。
- 2.使用gtest可以免去维护测试框架的麻烦，让我们有更多精力投入到案例设计上。
- 3.gtest提高了非常完善的功能，并且简单易用，极大的提高了编写测试案例的效率。

gtest的官方网站是：

<http://code.google.com/p/googletest/>

从官方的使用文档里，你几乎可以获得你想要的所有东西

<http://code.google.com/p/googletest/wiki/GoogleTestPrimer>

<http://code.google.com/p/googletest/wiki/GoogleTestAdvancedGuide>

如果还想对gtest内部一探究竟，就把它的代码下载下来研究吧，这就是开源的好处，哈！

官方已经有如此完备的文档了，为什么我还要写呢？一方面是自己记笔记，好记性不如烂笔头，以后自己想查查一些用法也可以直接在这里查到，一方面对于不想去看一大堆英文文档的朋友，在我这里可以快速的找到gtest相关的内容。

下面是该系列的目录：

- 1.玩转Google开源C++单元测试框架Google Test系列(gtest)之一 - 初识gtest
- 2.玩转Google开源C++单元测试框架Google Test系列(gtest)之二 - 断言
- 3.玩转Google开源C++单元测试框架Google Test系列(gtest)之三 - 事件机制
- 4.玩转Google开源C++单元测试框架Google Test系列(gtest)之四 - 参数化
- 5.玩转Google开源C++单元测试框架Google Test系列(gtest)之五 - 死亡测试
- 6.玩转Google开源C++单元测试框架Google Test系列(gtest)之六 - 运行参数
- 7.玩转Google开源C++单元测试框架Google Test系列(gtest)之七 - 深入解析gtest
- 8.玩转Google开源C++单元测试框架Google Test系列(gtest)之八 - 打造自己的单元测试框架

额外篇：

- 1.gtest中如何跳出当前测试案例
- 2.编写优美的GTest测试案例
- 3.gtest
参数化测试代码示例 (内含完整工程示例)

软件测试Linux环境中清爽的Google Test (GTest) 测试环境搭建【初级使用】

本文将介绍单元测试工具google test (GTEST) 在linux操作系统中测试环境的搭建方法。本文属于google test使用的基础教程。在linux中使用google test之前，需要对如下知识有一些了解：

- (1) C/C++编程方法
- (2) makefile的编写
- (3) linux命令行操作
- (4) GTEST背景知识：它是用来干什么的？它用什么语言编写的？它的一些基本语法？等等

以上知识不需要做到熟能生巧，只要达到入门级别，使用GTEST不会费任何力气，毕竟GTEST也不是一个高门槛的工具。言归正传，我们来按照步骤逐一讲解：

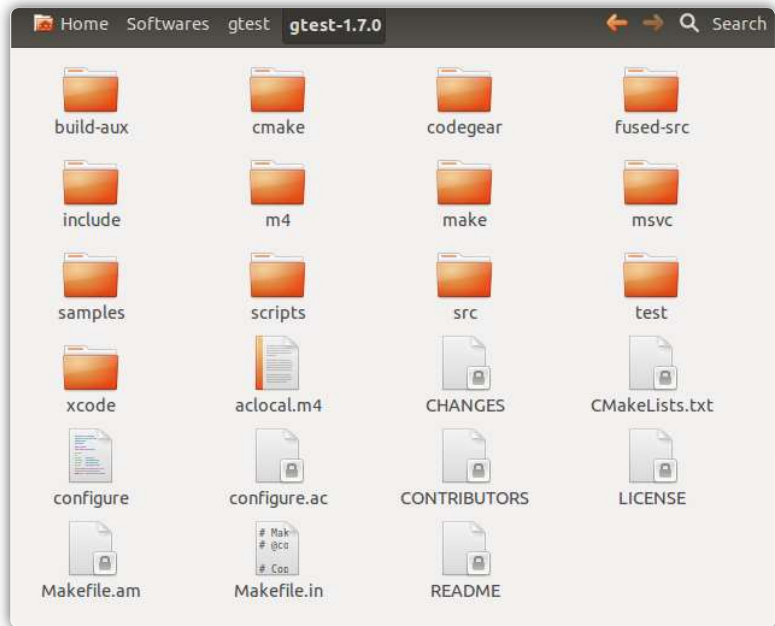
Step 1: 下载GTEST

下载链接为：<https://code.google.com/p/googletest/downloads/list>

目前GTEST的最新版本为gtest-1.7.0.zip，因此我们在本文中将以这个版本作为例子讲解。

Step 2: 认识文件夹

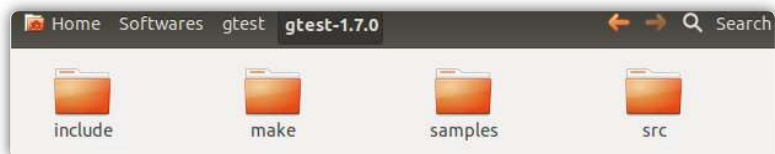
下载并解压之后，就可以看到gtest-1.7.0文件夹了，里面的内容如下图所示



这么多文件乍一看确实有点让人摸不到头脑。其实，GTEST提供了对于多个不同平台的支持，例如msvc文件夹是用在微软Visual Studio中，xcode文件夹是用于Mac Xcode，codegear文件夹是用于Borland C++ Builder，在linux环境中，我们用的内容就是make文件夹了。

Step 3: 清除不需要的文件

前一步中我们已经讲到，很多文件是为了支持不同平台的，为了保持程序的简洁，避免混淆试听，我们在这一步把所有不需要的文件全部删除，只保留我们需要的。其实我们只需要4个文件夹，如下图所示。其余的文件以及文件夹全部删除（这一步并不是必须，但对于我们有洁癖的程序员来讲，容不得一堆不用的代码放在那~）。



好了，只剩下了四个文件夹，看上去是不是清爽了很多？其实打开make文件夹，你会发现里面只有一个Makefile文件。查看Makefile文件内容，得知这是系统给出的编译samples文件夹中的第一个sample的命令。但是打开sample文件夹，又看到里面一大坨源文件。在本入门教程中，我们先不考虑那些复杂的例子。因此，打开samples文件夹，开始删文件，删到只剩下如下图所示的三个文件为止。

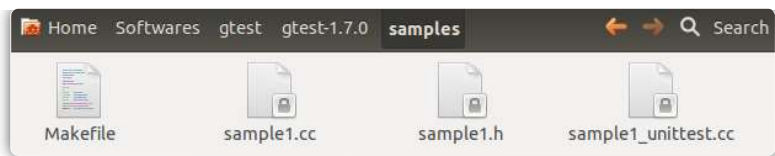


我们的程序越来越清爽了^_^。

Step 4: 改写Makefile文件

此时如果你到make文件夹下，通过命令行执行 `$ make && ./sample1_unittest` 命令，可以看到测试的执行结果。不过如果打开Makefile查看一下，就会发现这个makefile只适用于编译sample1，如果我再增加一个被测的源文件呢？又要重新写makefile，太麻烦了。于是，在这一步，我们改写一下Makefile。

上一步我们讲到，现在只剩下4个文件夹（include，make，samples，src），既然make里面的唯一一个文件也要被改写，那也没必要留这个文件夹了。于是，在Step 4，你要做的第一件事情就是，把make文件夹，连同里面的Makefile文件全部删除.....然后，进入samples文件夹，自己创建一个文件，名为Makefile，如图所示



然后，打开Makefile文件，写入以下内容，如图所示（不要把图中的行号也写进去哦~）。这个新的Makefile是由刚才被我们删除的Makefile改写而来的，如果你好奇的话可以比较一下它们之间的差别，里面涉及到一些makefile的语法和函数，如果不熟的话，你可能需要花费几分钟查一下资料去了解他们。

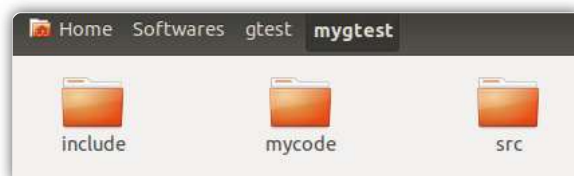
注意下图改写的Makefile第32行，我们编译的是后缀名为cpp的文件，而原来给的例子却以cc结尾。因此，你还要做一件事情，就是把sample1.cc的文件名改为sample1.cpp，把sample1_unittest.cc的文件名改为sample1_unittest.cpp，就大功告成了。

```

1 GTEST_DIR = ..
2 USER_DIR = .
3 CPPFLAGS += -isystem $(GTEST_DIR)/include
4 CXXFLAGS += -g -Wall -Wextra -pthread
5 TESTS = run_test
6 GTEST_HEADERS = $(GTEST_DIR)/include/gtest/*.h \
7                 $(GTEST_DIR)/include/gtest/internal/*.h
8 FILES = $(foreach d, $(USER_DIR), $(wildcard $(d)/*.cpp))
9 OBJS = $(patsubst %.cpp, %.o, $(FILES))
10
11
12 all : $(TESTS)
13
14 clean :
15     rm -f $(TESTS) gtest_main.a *.o
16
17 .PHONY : clean
18
19 GTEST_SRCS_ = $(GTEST_DIR)/src/*.cc $(GTEST_DIR)/src/*.h $(GTEST_HEADERS)
20
21 gtest-all.o : $(GTEST_SRCS_)
22     $(CXX) $(CPPFLAGS) -I$(GTEST_DIR) $(CXXFLAGS) -c \
23         $(GTEST_DIR)/src/gtest-all.cc
24
25 gtest_main.o : $(GTEST_SRCS_)
26     $(CXX) $(CPPFLAGS) -I$(GTEST_DIR) $(CXXFLAGS) -c \
27         $(GTEST_DIR)/src/gtest_main.cc
28
29 gtest_main.a : gtest-all.o gtest_main.o
30     $(AR) $(ARFLAGS) $@ $^
31
32 %.o : %.cpp
33     $(CXX) $(CPPFLAGS) $(CXXFLAGS) -c $< -o $@
34
35 $(TESTS) : $(OBJS) gtest_main.a
36     $(CXX) $(CPPFLAGS) $(CXXFLAGS) -lpthread $^ -o $@

```

现在我们的文件夹有三个（include，src，samples），我们自己被测的程序放在sample文件夹中。这个文件夹的名字看着也比较不爽，你可以把它改为mycode或者testcode，然后GTEST根目录的文件夹名称gtest-1.7.0也可以改为mygtest之类，用以满足我们完美主义者的需求。如图所示：



现在，进入命令行进行编译执行操作：\$ make && ./run_test，就可以看到结果了，如图所示：

```

Running main() from gtest_main.cc
[=====] Running 6 tests from 2 test cases.
[-----] Global test environment set-up.
[-----] 3 tests from FactorialTest
[ RUN    ] FactorialTest.Negative
[ OK     ] FactorialTest.Negative (0 ms)
[ RUN    ] FactorialTest.Zero
[ OK     ] FactorialTest.Zero (0 ms)
[ RUN    ] FactorialTest.Positive
[ OK     ] FactorialTest.Positive (0 ms)
[-----] 3 tests from FactorialTest (0 ms total)

[-----] 3 tests from IsPrimeTest
[ RUN    ] IsPrimeTest.Negative
[ OK     ] IsPrimeTest.Negative (0 ms)
[ RUN    ] IsPrimeTest.Trivial
[ OK     ] IsPrimeTest.Trivial (0 ms)
[ RUN    ] IsPrimeTest.Positive
[ OK     ] IsPrimeTest.Positive (0 ms)
[-----] 3 tests from IsPrimeTest (0 ms total)

[-----] Global test environment tear-down
[=====] 6 tests from 2 test cases ran. (0 ms total)
[ PASSED ] 6 tests.

```

Step 5: 添加自己的测试函数

到上面一步，其实我们的测试环境已经搭建完成。如果你现在有一个函数想要被测试一下，可以继续阅读Step 5。

假设我们现在有一个待测函数sqrt.cpp以及它的头文件sqrt.h，他们的内容如下：

(sqrt.cpp)

```
1 #include "sqrt.h"
2
3 int sqrt(int x) {
4     if(x<=0) return 0;
5     if(x==1) return 1;
6
7     int small=0;
8     int large=x;
9     int temp=x/2;
10
11     while(small<large){
12         int a = x/temp;
13         int b = x/(temp+1);
14
15         if (a==temp) return a;
16         if (b==temp+1) return b;
17
18         if(temp<a && temp+1>b){
19             return temp;
20         }
21         else if(temp<a && temp+1<b){
22             small=temp+1;
23             temp = (small+large)/2;
24         }else {
25             large = temp;
26             temp = (small+large)/2;
27         }
28     }
29     return -1;
30 }
```

(sqrt.h)

```
1 #ifndef _Sqrt_H_
2 #define _Sqrt_H_
3
4 int sqrt(int x);
5
6 #endif // _Sqrt_H_
```

(sqrt_unittest.cpp)

```
1 #include "sqrt.h"
2 #include "gtest/gtest.h"
3
4 TEST(SqrtTest, Zero){
5     EXPECT_EQ(0, sqrt(0));
6 }
7
8 TEST(SqrtTest, Positive){
9     EXPECT_EQ(100, sqrt(10000));
10    EXPECT_EQ(1000, sqrt(1000000));
11    EXPECT_EQ(99, sqrt(9801));
12 }
13
14 TEST(SqrtTest, Negative){
15     int i=-1;
16     EXPECT_EQ(0, sqrt(i));
17 }
18
```

这个被测文件的作用是计算任意一个正整数的平方根，算法复杂度在log(n)级别。将以上三个文件放在mycode文件夹中，然后 make

&& ./run_test进行编译运行，就可以看到结果了：

```

Running main() from gtest_main.cc
[=====] Running 9 tests from 3 test cases.
[-----] Global test environment set-up.
[-----] 3 tests from FactorialTest
[ RUN     ] FactorialTest.Negative
[ OK      ] FactorialTest.Negative (0 ms)
[ RUN     ] FactorialTest.Zero
[ OK      ] FactorialTest.Zero (0 ms)
[ RUN     ] FactorialTest.Positive
[ OK      ] FactorialTest.Positive (0 ms)
[-----] 3 tests from FactorialTest (0 ms total)

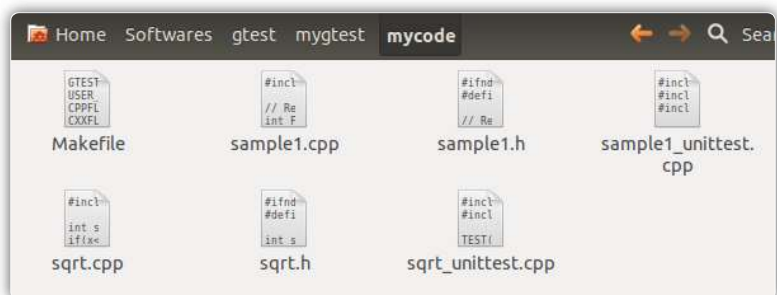
[-----] 3 tests from IsPrimeTest
[ RUN     ] IsPrimeTest.Negative
[ OK      ] IsPrimeTest.Negative (0 ms)
[ RUN     ] IsPrimeTest.Trivial
[ OK      ] IsPrimeTest.Trivial (0 ms)
[ RUN     ] IsPrimeTest.Positive
[ OK      ] IsPrimeTest.Positive (0 ms)
[-----] 3 tests from IsPrimeTest (0 ms total)

[-----] 3 tests from SQRTest
[ RUN     ] SQRTest.Zero
[ OK      ] SQRTest.Zero (0 ms)
[ RUN     ] SQRTest.Positive
[ OK      ] SQRTest.Positive (0 ms)
[ RUN     ] SQRTest.Negative
[ OK      ] SQRTest.Negative (0 ms)
[-----] 3 tests from SQRTest (1 ms total)

[-----] Global test environment tear-down
[=====] 9 tests from 3 test cases ran. (1 ms total)
[ PASSED ] 9 tests.

```

总结：环境搭建完成之后，每次测试一个文件xxx.cpp以及它的xxx.h文件，就把这俩放入mycode文件夹，然后编写xxx_unittest.cpp测试文件，也放进去。然后到这个目录下用命令行 make && ./run_test就可以了，应该比最开始的时候方便了许多吧？要测试时，我们只需要三个文件放入mycode，然后命令行进入这个目录 make && ./run_test 即可完成测试。下面的图就是我现在我的mycode文件夹了。xxx_unittest.cpp这个文件名并不是固定，取成别的也无所谓，只是这样更容易辨认哪一个文件是在测哪个函数而已。



当然，google test是一个非常强大的工具，以上所讲解的只是使用了它最基本的功能，以及一个简单环境的搭建。其实需要用的文件就是Step 4中提到的那剩下的三个文件夹内容，进行适当的改写，就可以实现更加强大的功能。这些等到以后有机会再写啦。

请赞赏

朋友，创作不易；为犒赏小编的辛勤劳动，请她喝杯咖啡吧！

给她赞赏，您将财运亨通



支付宝



微信

¥11 ¥13 **¥15** ¥20 ¥30

赞助商

感谢朋友，赞赏列表

我已完成赞赏，领取礼品

03-31 10:53

[Linux系统基础命令](#)

[#2020征文-开发板#基于TCP控制鸿蒙3861三色灯板亮灭\(附demo指导\)](#)

[ajr-Asynchronous JSON-RPC router.-Close Screen](#)

[什么是 Serverless 应用引擎？优势有哪些？](#)

[Javascript中的事件冒泡与捕获](#)

[mysql之视图，触发器，事务等。。。](#)

[正式发布专有云版本，阿里旗下低代码开发平台「宜搭」 如何发力政企数字化转型？](#)

[阅读源码，HashMap回顾](#)

[Hexo的详细搭建过程——小白的血泪经历QAQ](#)

[如何用git将项目代码上传到github](#)

[tp5 日志的用途以及简单使用](#)

[上传文件判断文件类型，添加文件的后缀名](#)

[mysql,按键精灵,读取写入](#)

[python基础语法20 面向对象5 exec内置函数的补充，元类，属性查找顺序](#)

[ReentrantLock源码学习总结 （二）](#)

[查看更多](#)